



Cours Fondement de l'intelligence artificielle (IA)

Chapitre 4: Jeux à deux adversaires

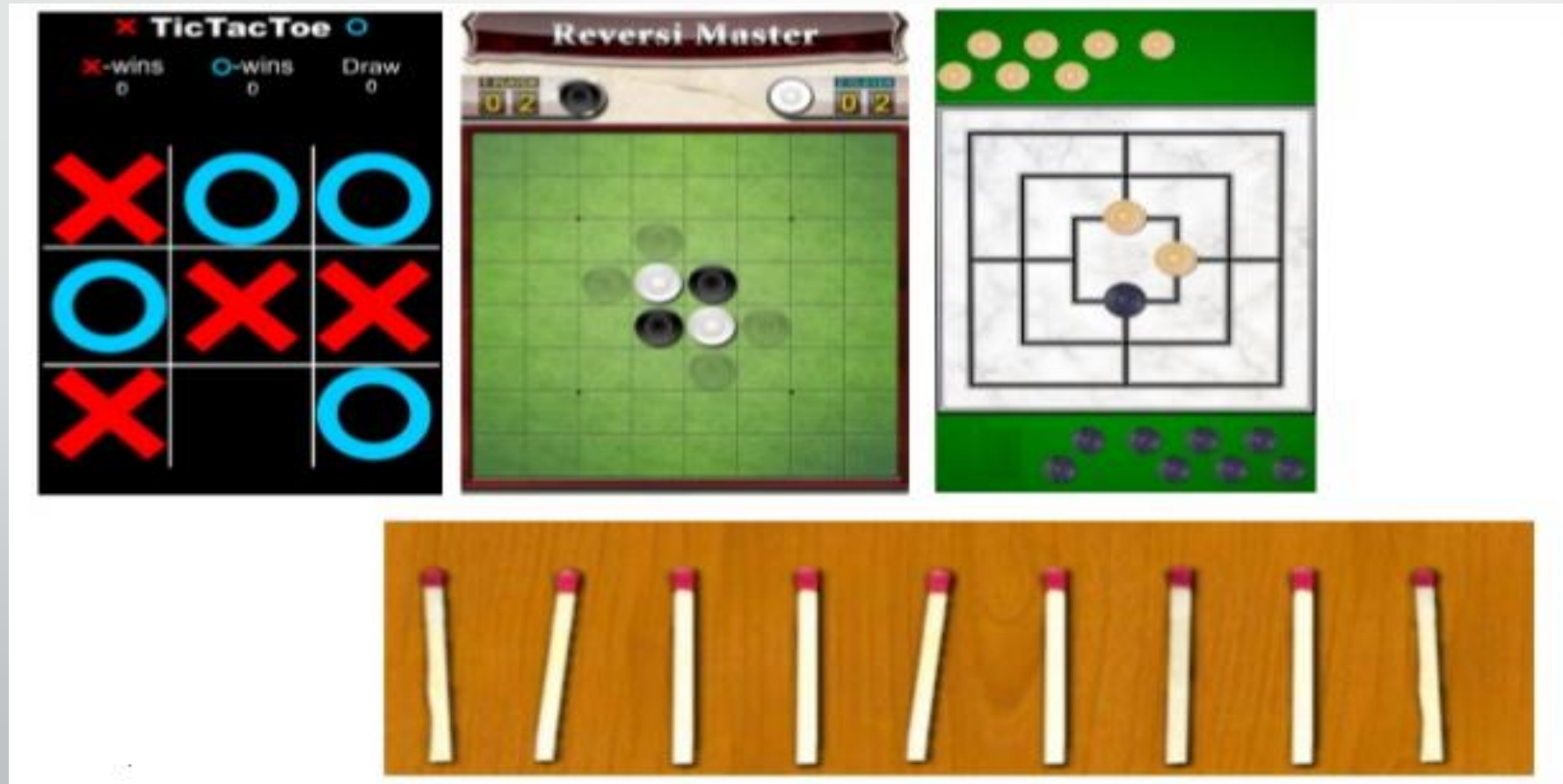
Année universitaire : 2025/2026

Plan

- Motivation
- Jeux à deux adversaires
- Algorithme Minimax
- Algorithme d'élagage α - β

Types de jeux

- Les jeux simples où une analyse exhaustive est possible: **Tic-Tac-Toe**, **Reversi**, **Moulin**, les **allumettes**



Types de jeux

- Des jeux plus complexes: dames, échecs, Othello, Awalé...
 - Des algorithmes spécifiques
 - Des méthodes heuristiques de recherche



Types de jeux

- Des jeux à information partielle: Bridge, Poker, Belote, Dominos.....
 - Des raisonnements de type probabiliste



Jeux à deux adversaires

- Caractéristiques
 - Les deux adversaires jouent à tour de rôle,
 - Jeux complètement observés (La situation globale du jeu est connue pour chacun des joueurs)
 - Jeux déterministes (la chance n'intervient pas)
 - Les jeux sont dits à **somme nulle**: les gains d'un joueur représentent les pertes à l'autre joueur

Jeux à deux adversaires

- Les jeux à deux adversaires et à informations complètes utilisent des techniques semblables à la représentation par graphe d'états (arbre):
 - Un nœud (état) initial: configuration initiale du jeu
 - Les nœuds de l'arbre: les configurations du jeu
 - Les branches: le passage d'une situation de jeu à une autre
- Un arbre de jeu représente le déplacement de deux adversaires

Arbre du jeu

- Du fait de la présence de deux joueurs ayant des objectifs antagonistes (opposés), la recherche de ces arbres ne peut se faire en utilisant des algorithmes de recherche de type A*
- Contrairement aux systèmes de résolution de problèmes, le planificateur ne dispose pas de la maîtrise complète de l'enchaînement des opérateurs, puisque des décisions extérieures à lui sont prises par l'adversaire.
- Il est possible, dans la plupart des jeux de générer l'ensemble de tout l'espace d'état
- Un arbre de jeu pour les dames, plus difficile que le Morpion, moins difficile que l'échec (10^{120} nœuds), est estimé à 10^{40} nœuds

Arbre de jeu

- Comme pour les problèmes que A* peut résoudre, on commence par déterminer la structure de notre espace d'état (de recherche)
- Un problème de jeu peut être vu comme un problème de recherche dans un arbre:
 - **Etat initial:** configuration initiale du jeu
 - **Fonction de transition:** cette fonction définit quelles actions sont possibles pour un joueur dans une configuration donnée, ainsi que les configurations résultantes des différentes actions.
 - **Test-but:** Ce test définit les configurations terminales du jeu
 - **Fonction d'utilité:** Cette fonction associe une valeur à chaque configuration terminale. Souvent, les valeurs sont +1, -1 et 0 qui correspondent respectivement à un gain pour le premier joueur, une perte pour le premier joueur et un match nul (parfois $+\infty$ et $-\infty$ sont utilisées au lieu de +1 et -1)

Arbre de jeu

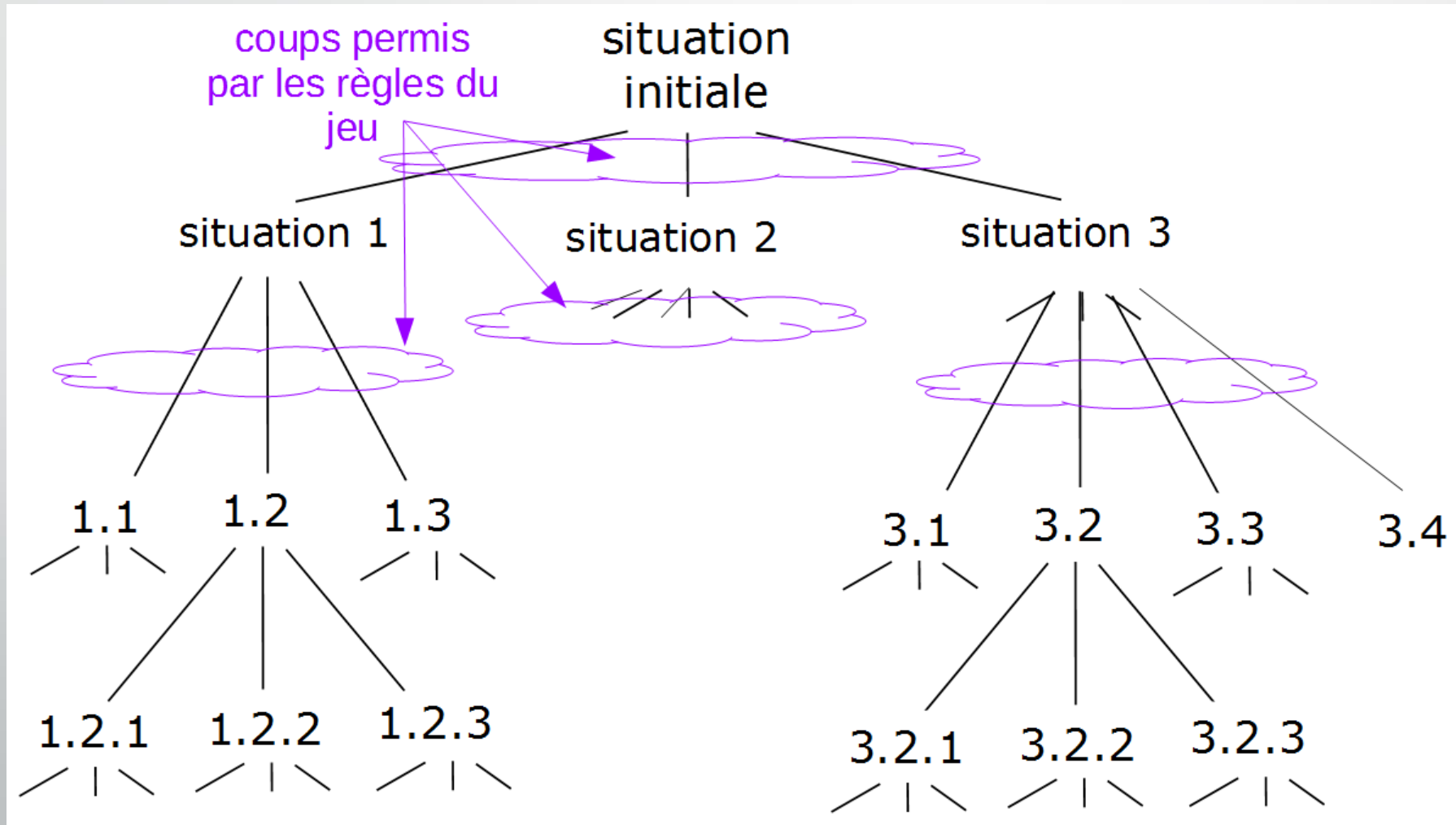
- **Remarques :**

- Notez que comme nous ne considérons que les jeux à somme nulle, nous n'avons besoin que d'une seule valeur par configuration: la valeur attribuée au deuxième joueur est exactement le négatif de la valeur reçue par le premier joueur.
- L'objectif du jeu pour un joueur est de maximiser son gain. Plus spécifiquement, le premier joueur veut que la valeur d'utilité de la configuration terminale du jeu soit la plus grande possible, et inversement, le deuxième joueur souhaite que cette valeur soit la plus petite possible.

Arbre de jeu

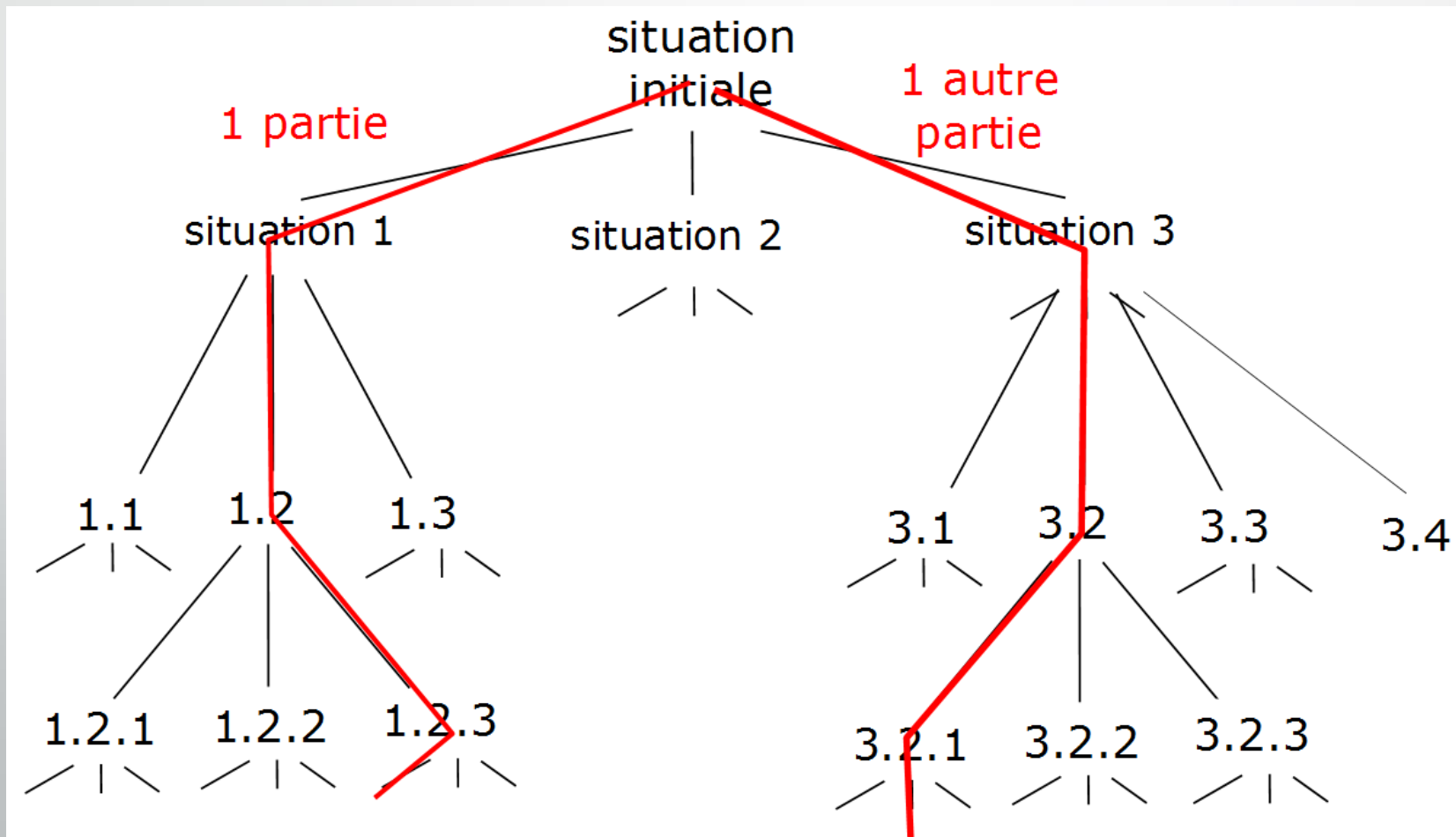
- On part d'une **situation courante** et les règles du jeu déterminent la liste des nouvelles situations de jeu atteignables à partir de celle-ci.
- Chacune de ces situations, si elle est choisie par le joueur, devient la prochaine **situation courante** à partir de laquelle l'autre joueur devra à son tour choisir la prochaine situation, parmi les situations permises, celle-ci devenant la **situation courante** du prochain tour de jeu et ainsi de suite jusqu'à la fin de la partie.
- Chaque **situation courante** propose donc un ensemble d'embranchements que peut choisir le joueur, et cette situation se reproduit à chaque tour de jeu.
- On peut donc représenter l'ensemble de toutes les parties possibles à partir d'une **situation courante** initiale sous la forme d'un arbre dont les nœuds sont les différentes situations de jeu possibles. Cet arbre est appelé **arbre de jeu**.

Arbre de jeu



Arbre de jeu

- La suite des choix des joueurs parmi les embranchements constituent la partie qu'ils jouent. Ce qui s'identifie par une branche de l'arbre de jeu. Chaque branche correspond à une partie différente.

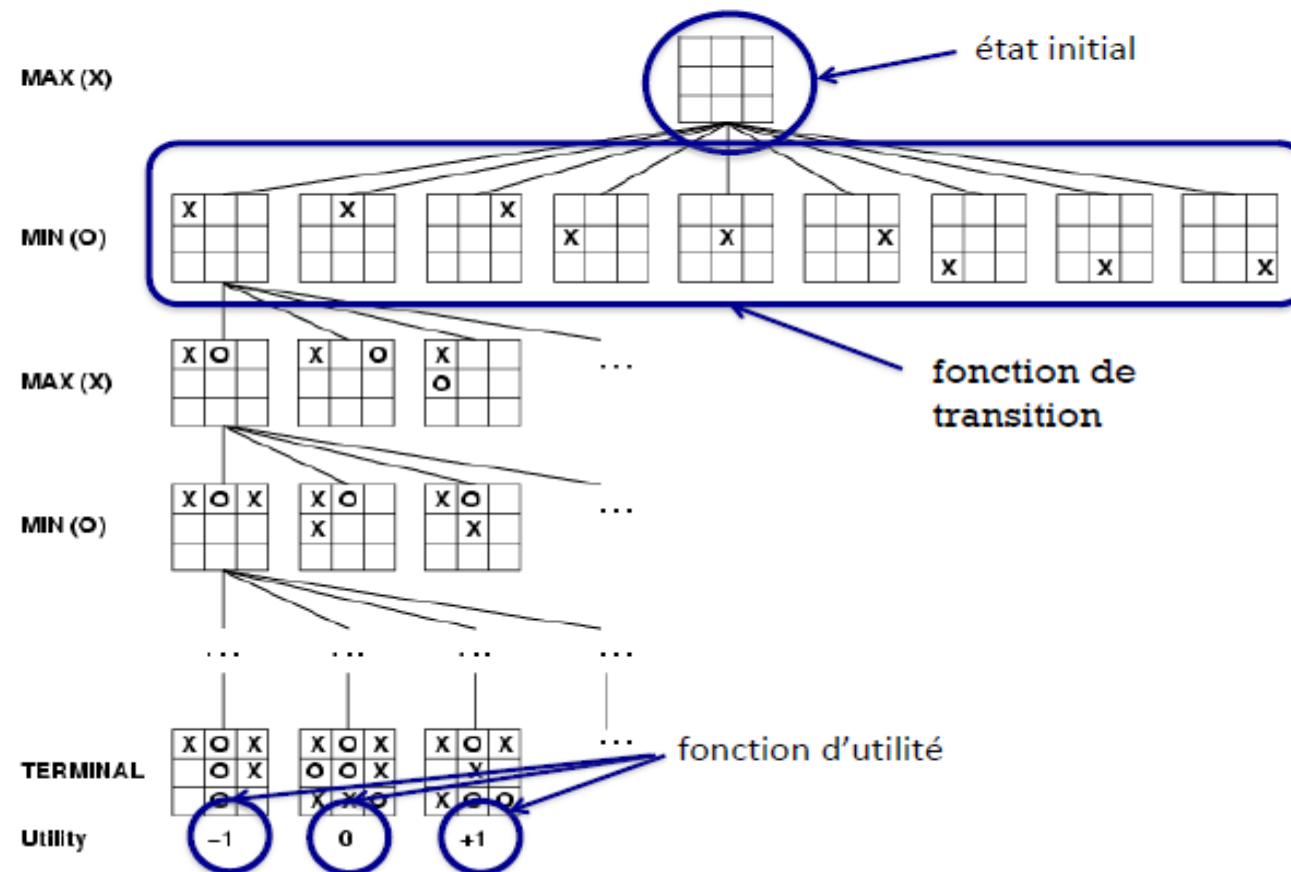


Exemple: jeu de TIC TAC TOE

- **Etat initial:** La grille est vide
- **Fonction de transition:** Quand c'est le tour du joueur X, il peut mettre un X dans une des cases vides de la grille. Même chose pour le joueur O.
- **Test-But:** Une configuration est terminale s'il y a trois X (ou trois O) sur une ligne, une colonne, ou une diagonale, ou si la grille est complète.
- **Fonction d'utilité:** Nous donnons une valeur +1 à toute configuration terminale contenant trois X dans une ligne, colonne ou diagonale (c'est X qui gagne) et -1 à toute configuration terminale avec trois O dans une même ligne, colonne ou diagonale (c'est O qui gagne). Les autres configurations terminales ont pour valeur 0, puisqu'elles correspondent à un match nul.

Exemple: jeu de TIC TAC TOE

ARBRE DE RECHERCHE TIC-TAC-TOE



Les méthodes de recherche des jeux

- Les problèmes de recherche, nous donnent une solution, qui est une suite d'actions reliant l'état initial et l'état but.
- Pour les jeux, une suite de coups n'est pas suffisante pour jouer parce que nous ne savons pas quels seront les coups de l'adversaire.
- Il faut donc trouver une stratégie qui définit comment le joueur doit jouer dans toutes les évolutions possibles du jeu. Stratégie consiste alors en un premier coup, puis un choix de coups pour chaque réponse possible de l'adversaire, puis un choix des coups pour les réponses à notre dernier coup et ainsi de suite.
- Une stratégie est optimale s'il n'existe pas d'autres stratégies qui donneraient un meilleur résultat contre un adversaire idéal

Les méthodes de recherche des jeux

- Algorithme Minimax
- Algorithme d'élagage α - β

L'algorithme Minimax

- On va noter par **MAX** le joueur qu'on cherche à faire gagner et son adversaire par **MIN**.
- Les deux joueurs désirent gagner le jeu.
- On suppose que le joueur **MIN** joue logiquement et qu'il ne va jamais rater une occasion de gagner.
- Si pour gagner, le joueur **MAX** essaie de maximiser son score, le joueur **MIN** désire aussi maximiser son propre score (ou de minimiser le score du joueur **MAX**).
- L'algorithme **MINIMAX**, dû à Von Neumann, a comme but l'élaboration d'une stratégie optimale pour le joueur **MAX**.
- À chaque tour le joueur **MAX** va choisir le coup qui va maximiser son score, tout en minimisant les bénéfices de l'adversaire. Ces bénéfices sont évalués en termes de la **fonction d'utilité** utilisée pour apprécier les positions pendant le jeu.
- L'arbre de jeu se transforme en un arbre avec une alternance de niveaux **MAX** et **MIN**.

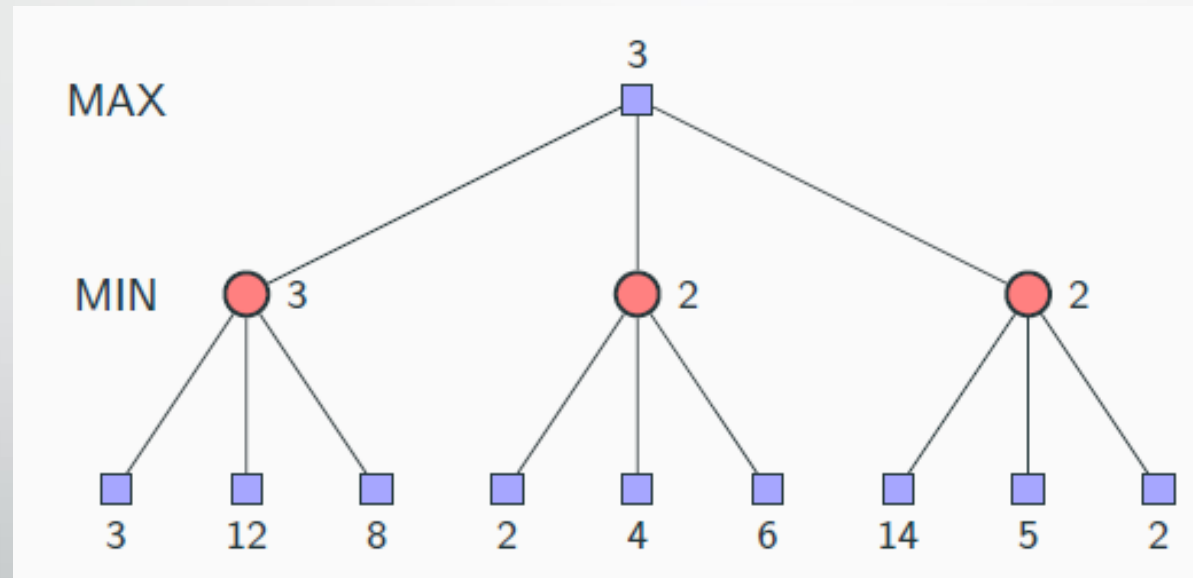
L'algorithme Minimax

- **Principe:**

- Une stratégie optimale peut être obtenue en examinant les valeurs minimax des nœuds de l'arbre de jeux.
- La valeur minimax d'un nœud donne la valeur d'utilité qui sera atteinte si chaque joueur joue de façon optimale à partir de la configuration de ce nœud. Pour calculer ces valeurs, on commence avec les configurations terminales, dont les valeurs minimax sont leurs valeurs d'utilité. Puis, on vérifie chaque nœud non terminal en utilisant les valeurs minimax de ces fils. La valeur minimax d'un nœud max sera le maximum des valeurs minimax de ses fils, parce que le joueur MAX veut maximiser la valeur de la configuration terminale.
- Pour les nœuds MIN, c'est l'inverse: nous prenons le minimum des valeurs minimax des ses fils car le joueur MIN cherche à obtenir une valeur minimale pour la configuration terminale

L'algorithme Minimax

- Exemple simple



L'algorithme Minimax

```
function MINIMAX-DECISION(state) returns an action
  inputs: state, current state in game
  return the a in ACTIONS(state) maximizing MIN-VALUE(RESULT(a, state))

function MAX-VALUE(state) returns a utility value
  if TERMINAL-TEST(state) then return UTILITY(state)
   $v \leftarrow -\infty$ 
  for a, s in SUCCESSORS(state) do  $v \leftarrow \text{MAX}(v, \text{MIN-VALUE}(s))$ 
  return v

function MIN-VALUE(state) returns a utility value
  if TERMINAL-TEST(state) then return UTILITY(state)
   $v \leftarrow \infty$ 
  for a, s in SUCCESSORS(state) do  $v \leftarrow \text{MIN}(v, \text{MAX-VALUE}(s))$ 
  return v
```

Algorithme d'élagage α - β

L'algorithme minimax visite chaque noeud de l'arbre de jeu, mais est-ce vraiment nécessaire ?

Dans le pire des cas, un parcours complet ne peut pas être évité, mais dans beaucoup d'autres cas, nous n'avons pas besoin d'examiner complètement l'arbre pour déterminer le meilleur coup.

Algorithme d'élagage α - β

- **idée : identifier des chemins dans l'arbre qui sont explorés inutilement**
- L'algorithme alpha-beta tire son nom des paramètres suivants décrivant les bornes des valeurs d'utilité enregistrée durant le parcours.
- α est la valeur du meilleur choix pour Max (c.-à-d., plus grande valeur) trouvé jusqu'ici.
- β est la valeur du meilleur choix pour Min (c.-à-d., plus petite valeur) trouvée jusqu'ici.

Algorithme d'élagage α - β

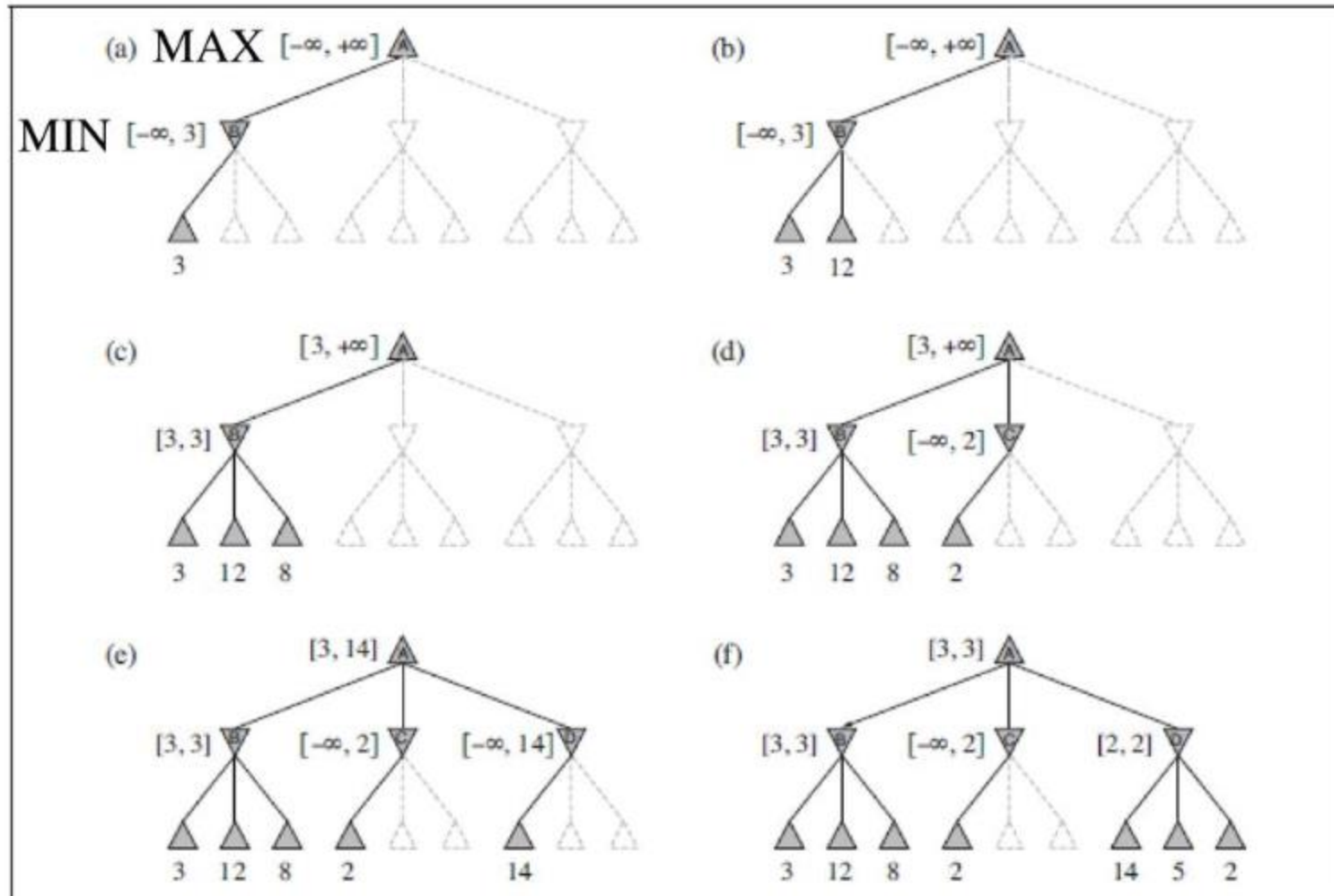
- Condition pour couper dans un nœud MIN
- Sachant que α est la valeur du meilleur choix pour Max (c.-à-d., plus grande valeur) trouvé jusqu'ici
- Si on est **dans un nœud Min et que sa valeur v devient inférieure à α (donc «pire que α » du point de vue de Max), il faut arrêter la recherche (couper la branche).**

Algorithme d'élagage α - β

- Condition pour couper dans un nœud MAX
- Sachant que β est la valeur du meilleur choix pour Min (c.-à-d., plus petite valeur) trouvé jusqu'ici
- Si on est **dans un nœud Max et que sa valeur devient supérieur à β (donc «pire que β » du point de vue de Max), il faut arrêter la recherche (*couper la branche*).**

Algorithme d'élagage α - β

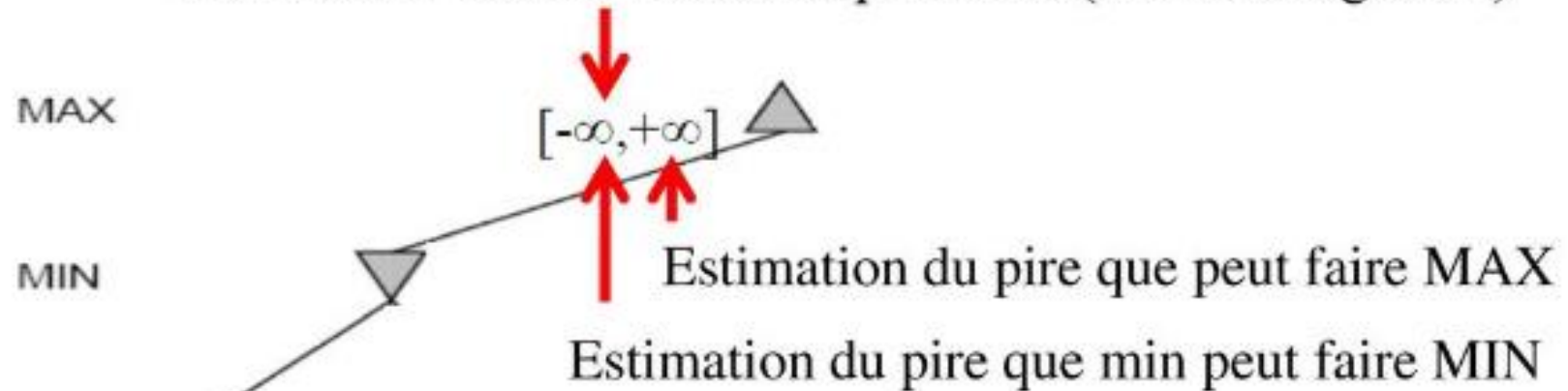
- Algorithme Alpha-Beta (exemple d'application des règles)



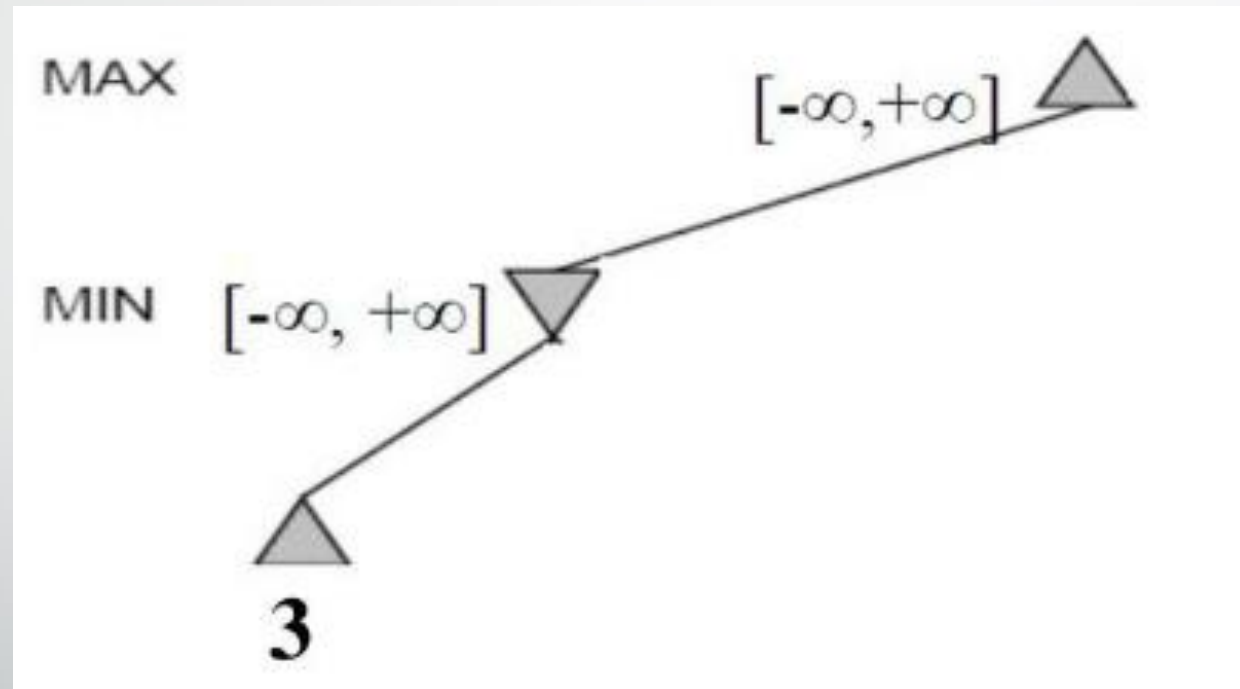
Algorithme d'élagage α - β

- Algorithme Alpha-Beta (exemple d'application des règles)

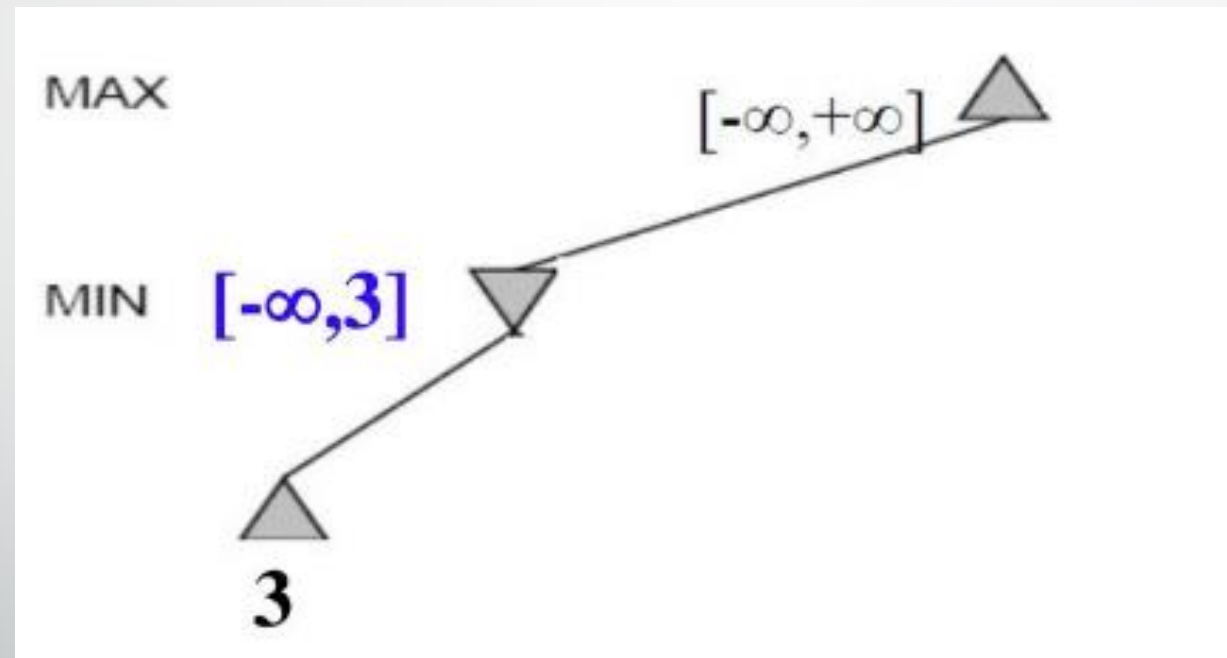
Bornes des valeurs MiniMax possibles (utilité atteignable)



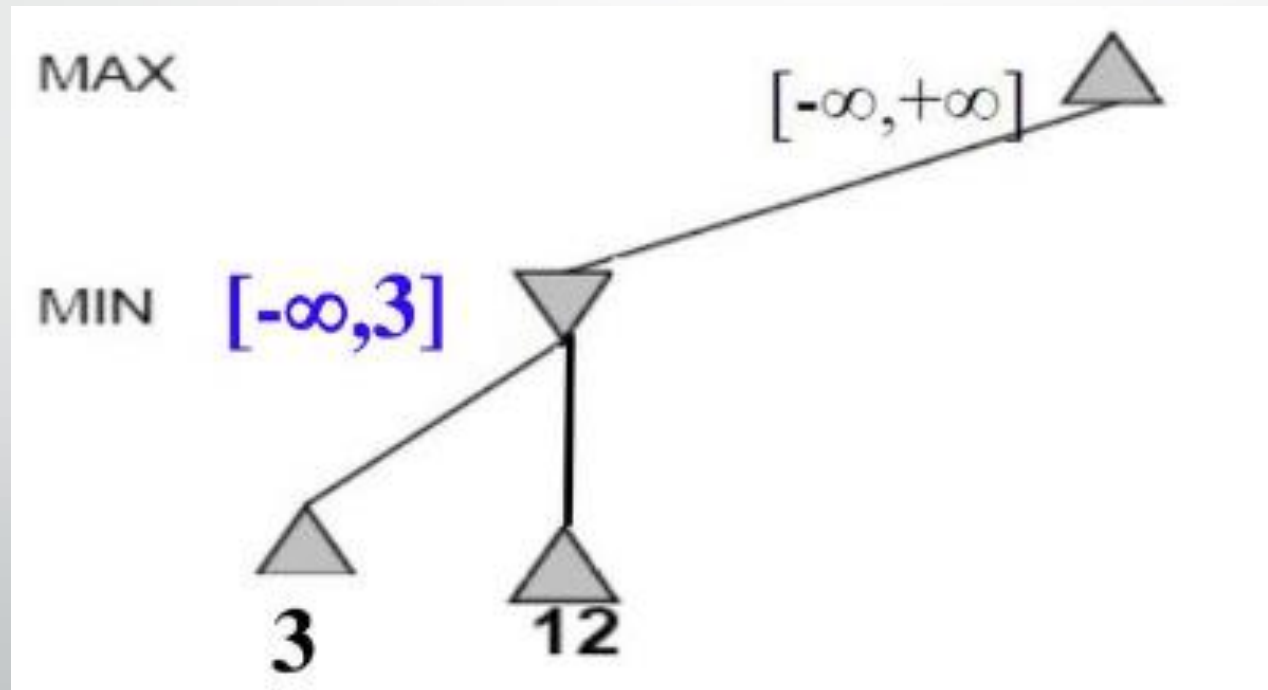
Algorithme d'élagage α - β



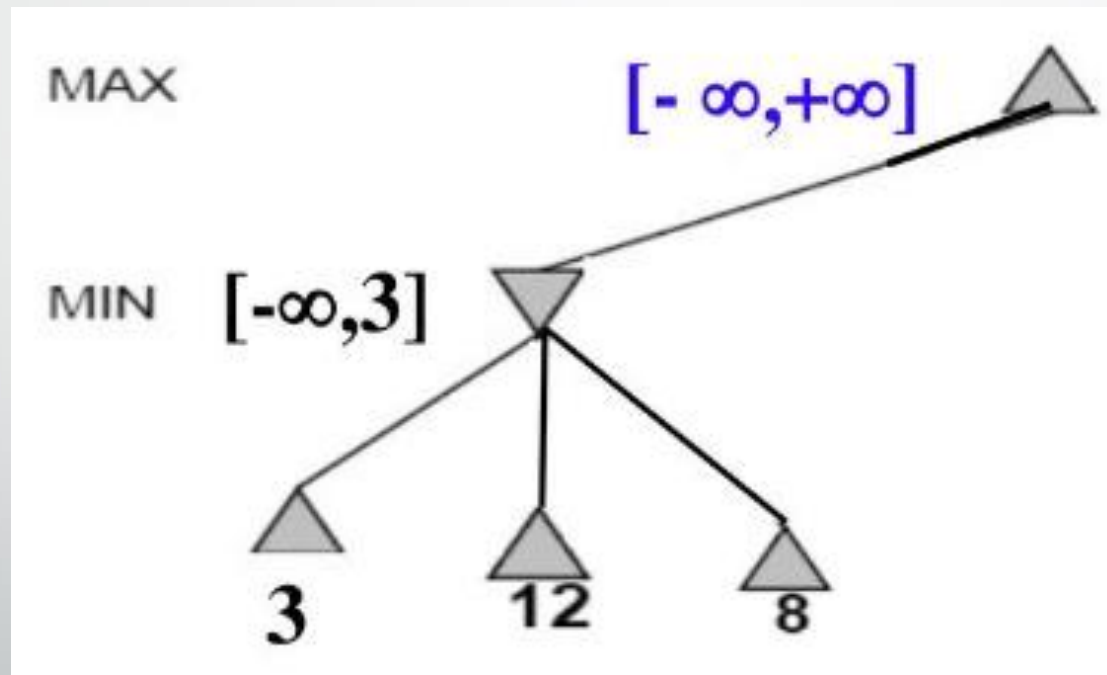
Algorithme d'élagage α - β



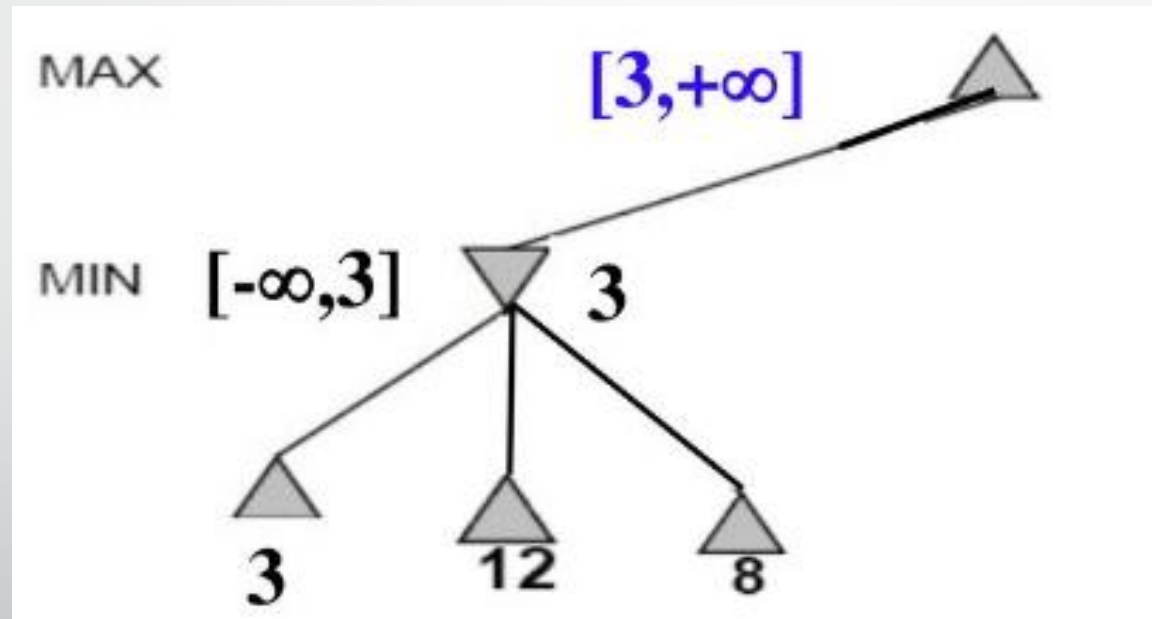
Algorithme d'élagage α - β



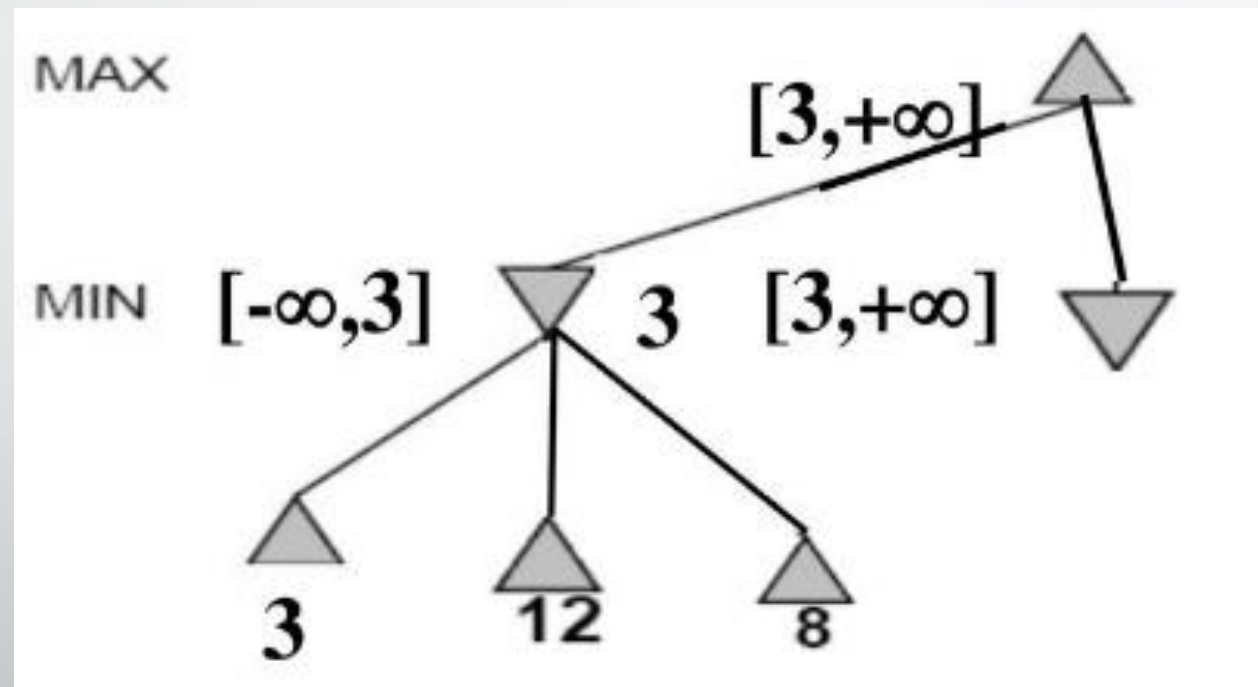
Algorithme d'élagage α - β



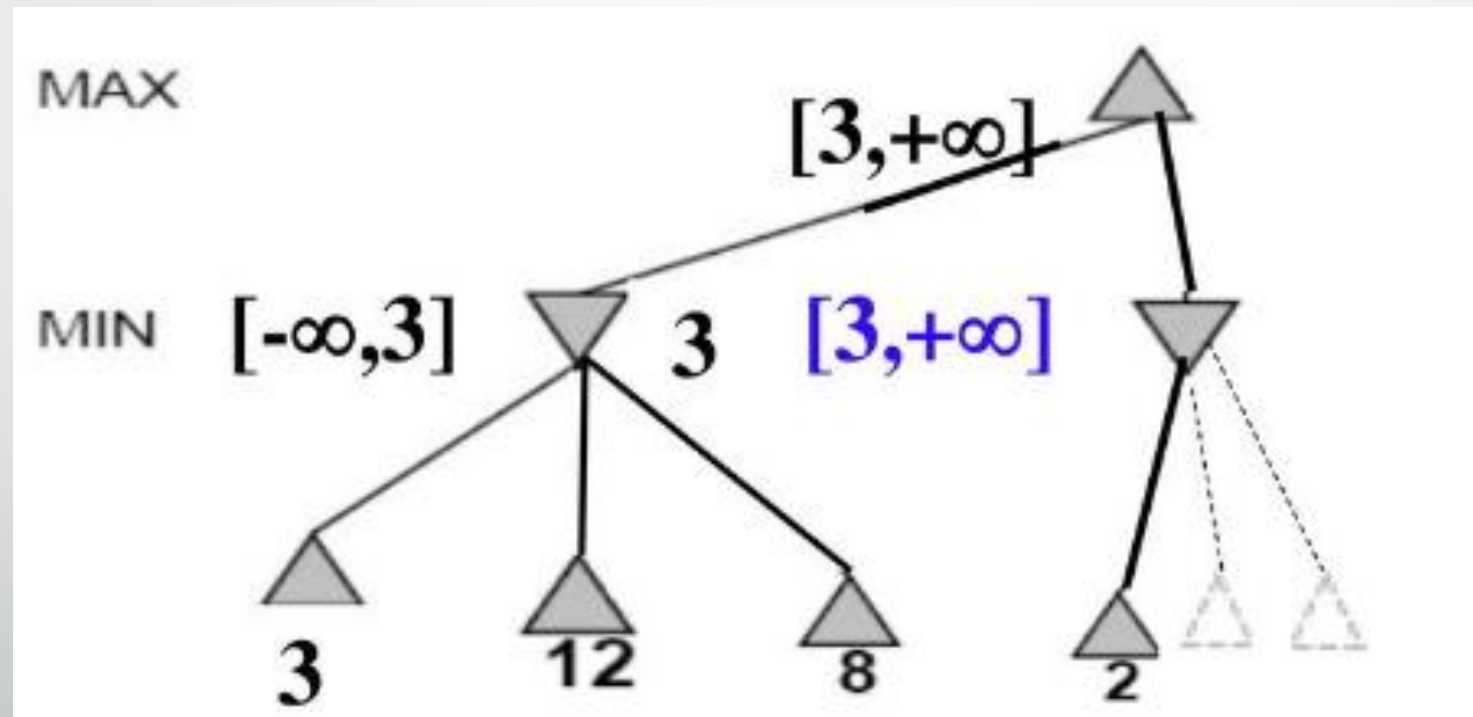
Algorithme d'élagage α - β



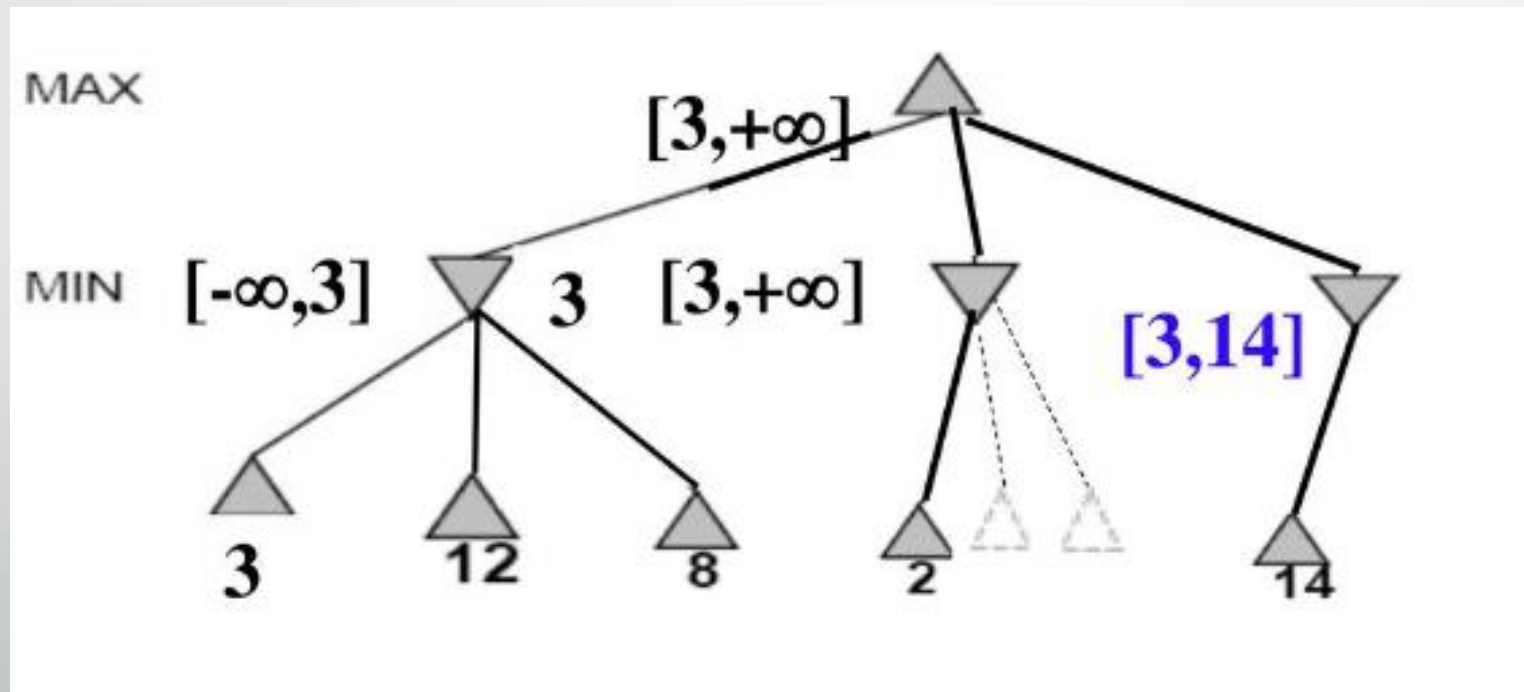
Algorithme d'élagage α - β



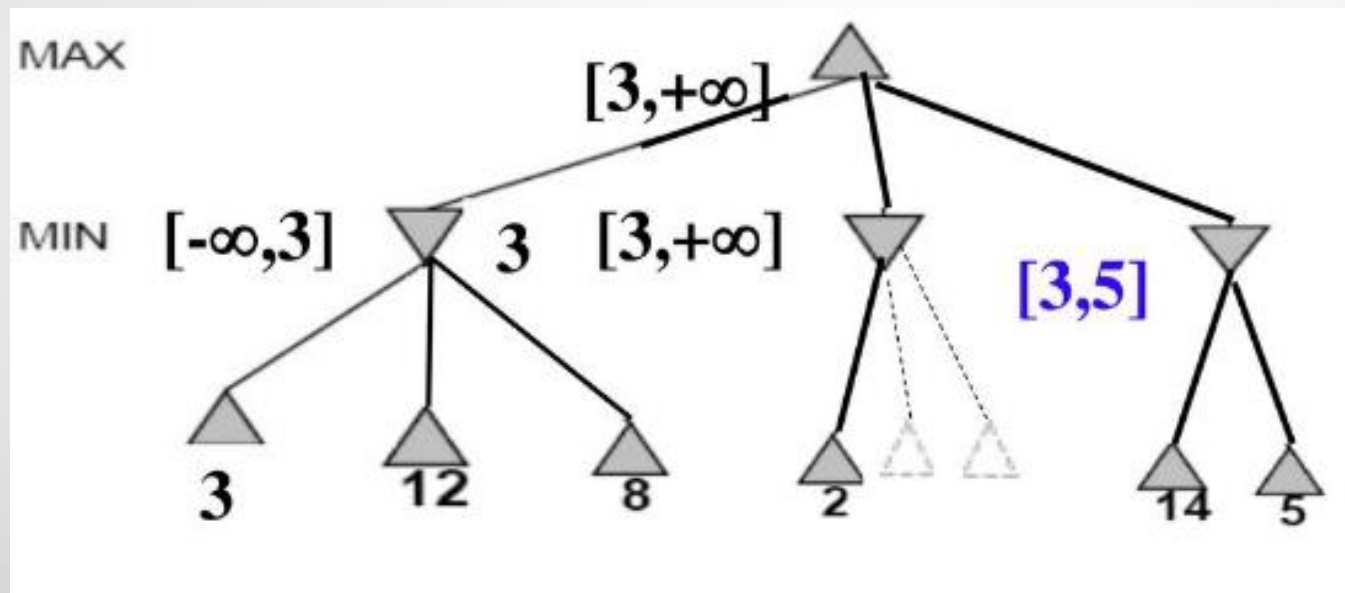
Algorithme d'élagage α - β



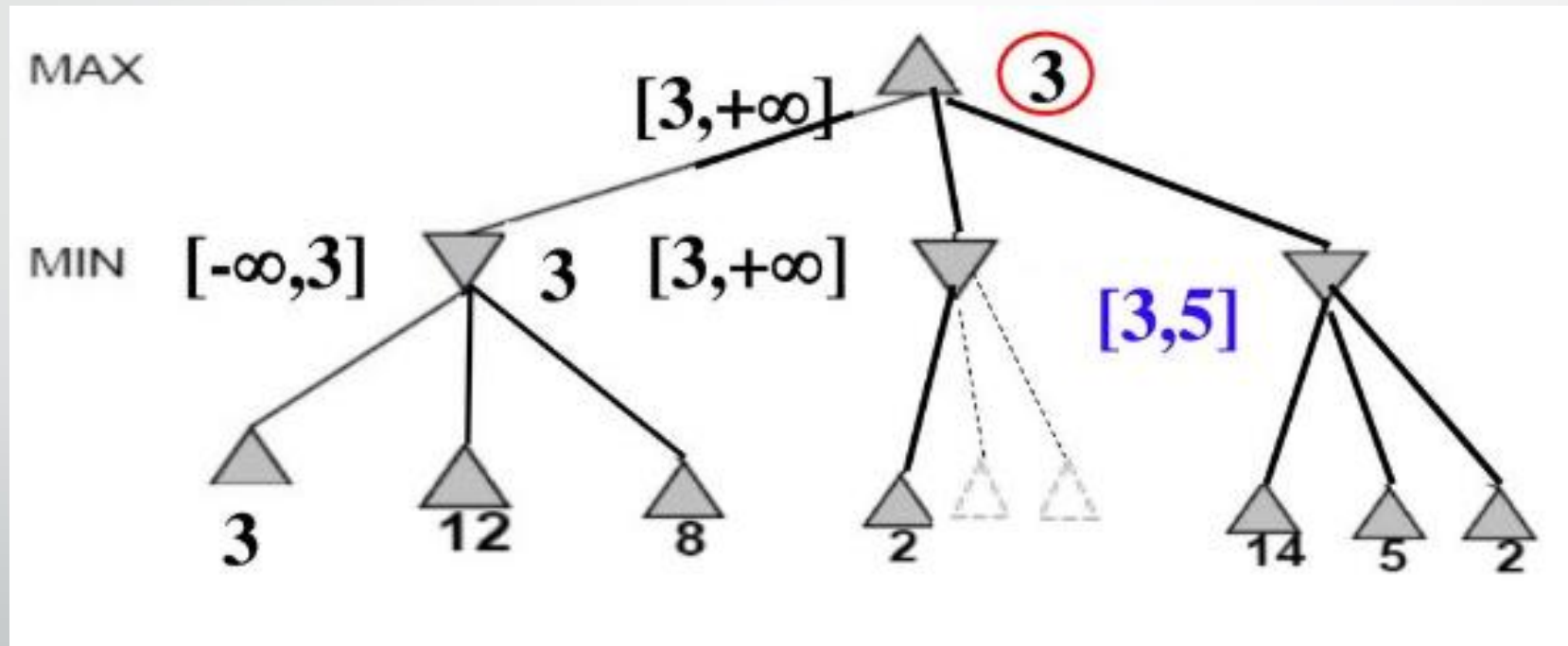
Algorithme d'élagage α - β



Algorithme d'élagage α - β



Algorithme d'élagage α - β



Algorithme d'élagage α - β

function ALPHA-BETA-SEARCH(*state*) *returns an action*

inputs: *state*, current state in game

$v \leftarrow \text{MAX-VALUE}(\text{state}, -\infty, +\infty)$

return the *action* in SUCCESSORS(*state*) with value v

function MAX-VALUE(*state*, α , β) *returns a utility value*

inputs: *state*, current state in game

α , the value of the best alternative for MAX along the path to *state*

β , the value of the best alternative for MIN along the path to *state*

if TERMINAL-TEST(*state*) **then return** UTILITY(*state*)

$v \leftarrow -\infty$

for a, s in SUCCESSORS(*state*) **do**

$v \leftarrow \text{MAX}(v, \text{MIN-VALUE}(s, \alpha, \beta))$

if $v \geq \beta$ **then return** v

$\alpha \leftarrow \text{MAX}(\alpha, v)$

return v

Algorithme d'élagage α - β

```
function MIN-VALUE(state,  $\alpha$ ,  $\beta$ ) returns a utility value
  inputs: state, current state in game
            $\alpha$ , the value of the best alternative for MAX along the path to state
            $\beta$ , the value of the best alternative for MIN along the path to state

  if TERMINAL-TEST(state) then return UTILITY(state)
   $v \leftarrow +\infty$ 
  for  $a, s$  in SUCCESSORS(state) do
     $v \leftarrow \text{MIN}(v, \text{MAX-VALUE}(s, \alpha, \beta))$ 
    if  $v \leq \alpha$  then return  $v$ 
     $\beta \leftarrow \text{MIN}(\beta, v)$ 
  return  $v$ 
```

Conclusion

- La recherche sur les jeux révèlent des aspects fondamentaux applicables à d'autres domaines
- La perfection est inatteignable dans les jeux : il faut approximer
- Dans le pire des cas, **Algorithme d'élagage α - β** se comporte comme **minimax** (explore tous les nœuds)
- Dans le meilleur cas, il peut résoudre un problème de profondeur 2 fois plus grande dans le même temps que minimax